Motivation for Normal Forms in Database Design

1. **Introduction to Normal Forms**

- In relational database design, **normal forms** are guidelines used to design schemas that reduce redundancy and dependency issues.

- The process of organizing data into normal forms is called **normalization**, which helps in ensuring the database is efficient and avoids anomalies.

2. **Key Problems Normalization Solves**

- **Redundancy**: Data duplication in a database leads to higher storage costs and inefficient data handling.

- **Update Anomalies**: If data appears in multiple places, updating in one location but forgetting others can cause inconsistencies.

- **Insertion Anomalies**: When certain attributes cannot be inserted without others, even if some values are unknown.

- **Deletion Anomalies**: Deleting data might inadvertently remove important information.

- Normal forms aim to structure the database in a way that these problems are minimized.

3. **What are Normal Forms?**

- A **normal form** is a set of criteria a database schema must meet.

- **First Normal Form (1NF)**: Ensures each column holds atomic (indivisible) values and that there are no repeating groups or arrays within a column.

- **Second Normal Form (2NF)**: Builds on 1NF by ensuring that every non-key attribute is fully dependent on the primary key.

- **Third Normal Form (3NF)**: Ensures that non-key attributes are not only dependent on the primary key but also directly dependent on it (i.e., no transitive dependencies).

- **Boyce-Codd Normal Form (BCNF)**: A stricter version of 3NF that handles certain types of functional dependencies more effectively.

4. **Benefits of Normalization**

- **Reduction in Data Redundancy**: Normalizing databases minimizes data duplication, leading to more efficient data storage.

- **Data Integrity**: By eliminating redundancy and enforcing constraints, normalization helps maintain consistent and reliable data.

- **Improved Update Efficiency**: Reduced redundancy means fewer places where changes need to be made, improving the efficiency of updates.

- **Clearer Data Relationships**: Normal forms structure data in a way that clearly defines relationships between tables, making the database easier to understand and work with.

5. **Consequences of Denormalized Databases**

- **Inconsistent Data**: Redundant copies of the same data can lead to inconsistencies if changes are made in one place but not others.

- **Wasted Storage**: Storing the same data multiple times unnecessarily increases the storage requirements.

- **Increased Processing Overhead**: More redundant data increases the complexity of database queries, making them slower to execute.

- **Difficult Maintenance**: Databases that are not normalized are harder to maintain because of the complexity introduced by redundancy.

6. **Trade-offs and Limitations**

- **Performance Considerations**: While normalization reduces redundancy, it can introduce complexity in queries that require data to be retrieved from multiple tables.

- **Denormalization for Performance**: In certain cases, databases may be denormalized intentionally to improve performance by reducing the need for complex joins.

7. **Use Cases for Normal Forms**

- **Transactional Databases**: Normal forms are crucial in transactional systems where data integrity and consistency are critical.

- **Dynamic Systems**: In systems where data is frequently updated or modified, normalized databases reduce the risk of anomalies.

- **Data Warehousing**: Though not as common in data warehousing (where denormalization may be preferred for performance), normalization helps in initial stages for data organization and cleansing.

8. **Conclusion**

- Normal forms are essential for creating efficient, scalable, and maintainable databases.

- They help database designers balance between storage efficiency and performance, ensuring the data remains consistent and free from anomalies.

- Depending on the application and performance needs, databases may either be fully normalized or partially denormalized.

Normalization forms the foundation of relational database theory, promoting clean and structured data design, which is vital for long-term scalability and efficiency.

Armstrong’s Axioms: Comprehensive Notes

1. **Introduction to Armstrong’s Axioms**

- **Armstrong's Axioms** are a set of rules used to infer all the functional dependencies in a database.

- These axioms are critical in the theory of **database normalization** and are used to find the closure of a set of functional dependencies.

- They provide a sound and complete method for deriving all possible functional dependencies from a given set of dependencies.

2. **Functional Dependencies (FDs)**

- A **functional dependency** is a relationship between two sets of attributes in a relational database.

- If `X → Y` is a functional dependency, it means that for any two tuples in a relation, if they agree on `X`, they must also agree on `Y`.

- **Example**: In a student database, if `StudentID → Name`, then for each unique `StudentID`, there is only one corresponding `Name`.

3. **Armstrong’s Axioms: Definition**

Armstrong's Axioms consist of three basic rules:

1. **Reflexivity Axiom (Trivial Dependency)**:

   - If `Y` is a subset of `X`, then `X → Y`.

   - **Explanation**: Any set of attributes functionally determines itself and any subset of itself.

   - **Example**: If `X = {A, B}`, then `X → A` and `X → B` are valid FDs because `{A}` and `{B}` are subsets of `X`.

2. **Augmentation Axiom (Partial Augmentation)**:

   - If `X → Y`, then `XZ → YZ` for any attribute set `Z`.

   - **Explanation**: If `X` functionally determines `Y`, adding another set of attributes `Z` to both sides does not affect the dependency.

   - **Example**: If `StudentID → Name`, then `StudentID, CourseID → Name, CourseID`.

3. **Transitivity Axiom (Transitive Dependency)**:

   - If `X → Y` and `Y → Z`, then `X → Z`.

   - **Explanation**: This is the transitive property. If `X` determines `Y`, and `Y` determines `Z`, then `X` determines `Z`.

   - **Example**: If `StudentID → Name` and `Name → Address`, then `StudentID → Address`.

4. **Derived Rules (Additional Axioms)**

Apart from the basic Armstrong's Axioms, several additional rules can be derived using these three axioms:

1. **Union Rule (Additivity)**:

   - If `X → Y` and `X → Z`, then `X → YZ`.

   - **Explanation**: If `X` determines both `Y` and `Z`, then `X` determines the combination of `Y` and `Z`.

   - **Example**: If `StudentID → Name` and `StudentID → Address`, then `StudentID → {Name, Address}`.

2. **Decomposition Rule (Projectivity)**:

   - If `X → YZ`, then `X → Y` and `X → Z`.

   - **Explanation**: If `X` determines a set of attributes `YZ`, it must individually determine `Y` and `Z`.

   - **Example**: If `StudentID → {Name, Address}`, then `StudentID → Name` and `StudentID → Address`.

3. **Pseudotransitivity Rule**:

- If `X → Y` and `YW → Z`, then `XW → Z`.

- **Explanation**: Combines the transitivity property with an additional attribute set `W`.

- **Example**: If `StudentID → Name` and `Name, Class → Grade`, then `StudentID, Class → Grade`.

5. **Uses of Armstrong's Axioms**

- **Closure of Functional Dependencies**:

- Armstrong's Axioms are used to find the **closure** of a set of functional dependencies, which is the set of all FDs implied by a given set.

- The closure helps database designers find all possible dependencies and verify whether a schema is in a certain normal form.

- **Minimal Cover**:

- Armstrong's Axioms are used to minimize a set of functional dependencies, creating a **minimal cover** where:

- Each dependency is in its simplest form (no unnecessary attributes).

- No dependencies can be removed without changing the overall meaning.

- **Checking for Equivalence**:

- Using these axioms, two sets of functional dependencies can be checked for **equivalence**, ensuring that both sets imply the same constraints.

6. **Examples of Using Armstrong's Axioms**

Example 1: Reflexivity

- Given: `X = {A, B, C}`, it is obvious that `X → A`, `X → B`, and `X → C` (because `A`, `B`, and `C` are subsets of `X`).

Example 2: Augmentation

- Given: `A → B`

- Augmenting with another attribute `C` on both sides, we get `AC → BC`.

Example 3: Transitivity

- Given: `A → B` and `B → C`

- By transitivity, we can infer `A → C`.

Example 4: Union

- Given: `A → B` and `A → C`

- We can infer `A → {B, C}` using the union rule.

7. **Importance of Armstrong’s Axioms in Database Design**

- **Soundness**: Armstrong's Axioms are **sound** because any functional dependency derived from these rules is valid.

- **Completeness**: They are also **complete**, meaning that all possible functional dependencies can be derived using these rules.

- **Schema Design**: By systematically applying Armstrong’s Axioms, database designers can ensure that a schema is properly normalized, reducing redundancy and preventing anomalies.

8. **Closure Algorithm Using Armstrong’s Axioms**

- To find the closure of a set of functional dependencies, apply Armstrong’s Axioms iteratively until no more new functional dependencies can be generated.

- **Steps**:

  1. Begin with the initial set of FDs.

  2. Apply the axioms (reflexivity, augmentation, transitivity) to derive new FDs.

  3. Continue applying the axioms until no further FDs can be derived (closure).

9. **Conclusion**

- Armstrong's Axioms provide a formal and systematic way to reason about functional dependencies in databases.

- They are fundamental to the process of **database normalization**, ensuring that schemas are free from redundancy and dependency anomalies.

- Mastery of Armstrong's Axioms allows for a deeper understanding of how to design efficient and consistent databases.

Closure of a Set of Functional Dependencies: Comprehensive Notes

1. **Introduction**

- In relational database design, the **closure** of a set of functional dependencies is an essential concept for analyzing the relationships between attributes.

- The closure of a set of functional dependencies allows database designers to determine all possible functional dependencies that can be implied from a given set.

- **Functional dependencies (FDs)** describe the relationships between attributes in a relation. If `X → Y`, this means the values of the attribute(s) `X` uniquely determine the values of `Y`.

2. **Definition of Closure**

- The **closure** of a set of functional dependencies, denoted as **F^+**, is the set of all functional dependencies that can be logically derived from a given set `F` of functional dependencies.

- In other words, if you are given a set `F` of FDs, the closure `$F^+$` contains all FDs that are logically implied by the FDs in `F`.

3. **Importance of Closure**

- **Schema Design**: Closure is critical in database normalization, as it helps determine whether certain dependencies exist between attributes.

- **Checking Dependency Satisfaction**: By finding the closure, one can verify whether a certain functional dependency holds in a relation.

- **Candidate Key Identification**: Closure helps in finding **candidate keys**, which are minimal sets of attributes that can uniquely identify tuples in a relation.

- **Normalization**: Closure is used to test if a relation meets specific normal forms like 2NF, 3NF, or BCNF, by determining whether certain FDs exist or not.

4. **Finding the Closure of a Set of Attributes (X^+)**

The closure of a set of attributes `X` under a set of functional dependencies `F` is the set of all attributes that are functionally determined by `X`.

Algorithm to Find Closure of Attribute Set X (X^+):

- **Input**: A set of attributes `X` and a set of functional dependencies `F`.
- **Output**: The closure `$X^+$` of `X` under `F`.

Steps:

1. **Initialize**: Start with `$X^+ = X$`, the closure initially contains the set `X`.
2. **Apply FDs**: For each functional dependency `$A \rightarrow B$` in `F`, if `$A \subseteq X^+$`, then add `B` to `$X^+$`.
3. **Repeat**: Repeat the process of adding attributes until no more new attributes can be added to `$X^+$`.
4. **Termination**: The process terminates when no more new attributes can be added to `$X^+$`.

Example:

- Given:
  - Set of attributes: `$X = \{A\}$`
  - Set of functional dependencies `$F = \{A \rightarrow B, B \rightarrow C\}$`
- Find `$X^+$`:
  - Start with `$X^+ = \{A\}$`
  - From `$A \rightarrow B$`, add `B` to `$X^+$`. Now, `$X^+ = \{A, B\}$`.
  - From `$B \rightarrow C$`, add `C` to `$X^+$`. Now, `$X^+ = \{A, B, C\}$`.
  - No more attributes can be added, so `$X^+ = \{A, B, C\}$`.

5. **Finding the Closure of a Set of Functional Dependencies (F^+)**

To find the closure of a set of functional dependencies `F`, you use **Armstrong's Axioms** to derive all possible functional dependencies from the given set.

Steps to Find F^+:

1. **Start with F**: Begin with the given set of functional dependencies `F`.

2. **Apply Armstrong's Axioms**: Use Armstrong's Axioms (Reflexivity, Augmentation, and Transitivity) to derive new functional dependencies.

3. **Repeat**: Continue applying the axioms until no new functional dependencies can be derived.

4. **Termination**: The process terminates when all possible dependencies have been derived. The resulting set is the closure F^+.

Armstrong's Axioms:

- **Reflexivity**: If $Y \subseteq X$, then $X \rightarrow Y$.

- **Augmentation**: If $X \rightarrow Y$, then $XZ \rightarrow YZ$ for any attribute set Z.

- **Transitivity**: If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.

Example:

- Given functional dependencies:

  - $F = \{A \rightarrow B, B \rightarrow C\}$

- Find F^+:

  - Start with the given FDs.

  - From $A \rightarrow B$ and $B \rightarrow C$, by transitivity, we can infer $A \rightarrow C$.

  - The closure $F^+ = \{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$.

6. **Application of Closure**

a. Candidate Key Identification

- A **candidate key** is a minimal set of attributes that can uniquely identify a tuple in a relation.

- To find a candidate key, compute the closure of various attribute sets and check if the closure includes all attributes in the relation.

- If X^+ contains all attributes, then X is a superkey.

- If X is minimal (i.e., no subset of X can also be a superkey), then X is a candidate key.

Example:

- Relation $R = \{A, B, C, D\}$ with FDs: $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$

- Find candidate key:
  - Compute the closure of `A`:
    - `$A^+$ = {A, B, C, D}` (since `A → B`, `B → C`, and `C → D`).
    - Therefore, `A` is a candidate key.

b. Testing Functional Dependencies

- Closure is used to test if a specific functional dependency holds in a relation.
- To check if `X → Y` holds, compute `$X^+$` under the set of FDs and check if `Y` is included in `$X^+$`.

Example:

- Given `F = {A → B, B → C}`, check if `A → C` holds:
  - Compute `$A^+$ = {A, B, C}`.
  - Since `C` is in `$A^+$`, the dependency `A → C` holds.

c. Decomposition and Dependency Preservation

- Closure helps in testing whether the decomposition of a relation preserves functional dependencies.
- If the FDs in a decomposed schema are present in the closure of the original FDs, then the decomposition is dependency-preserving.

7. **Example of Closure Computation**

Example:

- Relation `R(A, B, C, D, E)`
- Functional Dependencies: `F = {A → B, B → C, A → D, D → E}`
- Compute closure of `{A}`:
  - Start with `$A^+$ = {A}`
  - From `A → B`, add `B` to `$A^+$`: `$A^+$ = {A, B}`
  - From `B → C`, add `C` to `$A^+$`: `$A^+$ = {A, B, C}`
  - From `A → D`, add `D` to `$A^+$`: `$A^+$ = {A, B, C, D}`

- From `D ⟶ E`, add `E` to `A⁺`: `A⁺ = {A, B, C, D, E}`

- Closure of `{A}` is `{A, B, C, D, E}`.

8. **Conclusion**

- The closure of a set of functional dependencies is crucial in relational database design, enabling the derivation of all implied functional dependencies.

- It helps in key tasks such as identifying candidate keys, verifying FDs, and ensuring dependency preservation during normalization.

- Armstrong's Axioms provide a systematic approach for computing closure, ensuring that all possible dependencies are accounted for. Understanding and calculating closures is fundamental for designing efficient and well-structured database schemas.

Definitions of 1NF and 2NF in Database Normalization

1. **First Normal Form (1NF)**

Definition:

A relation is in **First Normal Form (1NF)** if it satisfies the following conditions:

- All attributes (columns) contain **atomic values**; that is, each cell in a table contains a single, indivisible value.

- There are **no repeating groups** or arrays within a row. This means that no column should have multiple values (like lists or sets).

- The order in which data is stored does not matter, but the data must be well-structured into rows and columns.

Key Characteristics:

- Every column must have atomic values (no nested tables or multiple values in a single column).

- There should be a **primary key** that uniquely identifies each row.

- **No duplication** of rows is allowed.

Example (Violation of 1NF):

| StudentID | Name | Subjects |

|---|---|---|

| 1 | John | Math, Physics |

| 2 | Alice | Chemistry, Biology |

Corrected (in 1NF):

| StudentID | Name | Subject |

|---|---|---|

| 1 | John | Math |

| 1 | John | Physics |

| 2 | Alice | Chemistry|

| 2 | Alice | Biology |

In 1NF, we split the multi-valued `Subjects` into separate rows, ensuring atomic values in each cell.

---

2. **Second Normal Form (2NF)**

Definition:

A relation is in **Second Normal Form (2NF)** if:

- It is already in **1NF**.

- Every non-key attribute is **fully functionally dependent** on the entire primary key. This means there should be **no partial dependency** (i.e., no non-prime attribute should depend on only a part of a composite key).

Key Characteristics:

- If the primary key is **composite** (i.e., made up of more than one attribute), each non-key attribute must be dependent on **all** parts of the composite key, not just a subset.

- If a non-key attribute is dependent on only a part of the composite key, it violates 2NF and needs to be decomposed.

Example (Violation of 2NF):

StudentID	CourseID	CourseName	Grade
1	101	Math	A
1	102	Physics	B
2	101	Math	A

In this case, `CourseName` is dependent only on `CourseID` and not on the full composite key (`StudentID, CourseID`), which violates 2NF.

Corrected (in 2NF):

- Split into two tables:

Student-Course Relation:

StudentID	CourseID	Grade
1	101	A
1	102	B
2	101	A

Course Relation:

CourseID	CourseName
101	Math
102	Physics

Now, `CourseName` is fully dependent on `CourseID`, and the remaining relation (`StudentID, CourseID → Grade`) is in 2NF.

Summary:

- **1NF** eliminates multi-valued attributes and ensures atomicity.

- **2NF** eliminates partial dependencies by ensuring non-key attributes depend on the entire primary key (especially when it’s composite).

Multivalued Dependency

What is Multivalued Dependency in DBMS?

Multivalued dependency would occur whenever two separate attributes in a given table happen to be independent of each other. And yet, both of these depend on another third attribute. The multivalued dependency contains at least two of the attributes dependent on the third attribute. This is the reason why it always consists of at least three of the attributes.

Example

Suppose that there is a car manufacturing company that produces two of the colours in the market, i.e., red and yellow of each of their models, every year.

CAR_MODEL	MANUF_MONTH	COLOUR
S2011	JAN	Yellow
S2001	FEB	Red
S3001	MAR	Yellow
S3001	APR	Red
S4006	MAY	Yellow
S4006	JUN	Red

In this case, the columns COLOUR and MANUF_MONTH are dependent on CAR_MODEL, and they are independent of each other. Thus, we can call both of these columns multivalued. These are, as a result, dependent on CAR_MODEL. Here is a representation of the dependencies we discussed above:

CAR_MODEL → → MANUF_MONTH

CAR_MODEL → → COLOUR

We can read this as “CAR_MODEL multidetermined MANUF_MONTH” and “CAR_MODEL multidetermined COLOUR”.

Conditions for Multivalued Dependency in DBMS

An MVD would mean that for some single value of the attribute ‘x’, multiple values of attribute ‘y’ can exist. Thus, we will write it as follows:

x –> –> y

So, it is actually read as: x is multivalued dependent only.

Now, let us suppose that a student named Rita is working on two projects, Oracle and Microsoft and has two separate hobbies, namely Music and Reading. We can easily express this data in some tabular format in the way:

	a	b	c
	Name	Project	Hobby
p1	Rita	MS	Reading
p2	Rita	Oracle	Music
p3	Rita	MS	Music
p4	Rita	Oracle	Reading

Here project and hobby are multivalued attributes because they contain different values for the same name (rita)

Attributes(columns) : a, b, c
Tupples(rows) : p1, p2, p3, p4

Q = set of attributes q = relation

Conditions for MVD:

An attribute say x can define another attribute y; if a legal relation q(Q), for all the pairs of tuples p1 and p2 in q, such that,

p1[a] = p2[a]

Then there exists p3 and p4 in r such that.

p1[a] = p2[a] = p3[a] = p4[a]

p1[b] = p3[b]; p2[b] = p4[b]

p1 = p4; p2 = p3

Thus, MVD exists in this case.

In order to check the MVD in the given table, we will apply the conditions that were stated above, and we will then check it with the values present in the given table.

	a	b	c
	Name	Project	Hobby
p1	Rita	MS	Reading
p2	Rita	Oracle	Music
p3	Rita	MS	Music
p4	Rita	Oracle	Reading

Condition-1:

p1[a] = p2[a] = p3[a] = p4[a]

Finding from table,

p1[a] = p2[a] = p3[a] = p4[a] = Rita

Thus, condition 1 is satisfied here.

Condition-2:

p1[b] = p3[b]

And

p2[b] = p4[b]

Finding from table,

p1[b] = p3[b] = MS

And

p2[b] = p4[b] = Oracle

Thus, condition 2 is satisfied here.

Condition-3:

p1 = p4

And

p2=p3
Finding from table,
p1 = p4 = Reading
And
p2 = p3 = Music
Thus, condition 3 is satisfied here.
All the conditions here are satisfied, thus,
a –> –> b
According to table,
name –> –> project
And for,
a –> –> C
We will get,
name –> –> hobby
Thus, we know that MVD would exist in the table given above, and it can be stated like,
name –> –> project
name –> –> hobby

Joint Dependency

What is Join Dependency in DBMS?

Whenever we can recreate a table by simply joining various tables where each of these tables consists of a subset of the table's attribute, then this table is known as a Join Dependency. Thus, it is like a generalization of MVD. We can relate the JD to 5NF. Herein, a relation can be in 5NF only when it's already in the 4NF. Remember that it cannot be further decomposed.

Examples of Join Dependency in DBMS

Example 1

<Student>

Stu_Name	Stu_Skills	Stu_Job (Assigned Work)
Tag	Marketing	GK001
Barry	PR	GK002
Paulo	Graphic Designing	GK003

We can decompose the table given above into these three tables given below. And thus, it is not in the Fifth Normal Form.

<Student_Skills>

Stu_Name	Stu_Skills
Tag	Marketing
Barry	PR
Paulo	Graphic Designing

<Student_Job>

Stu_Name	Stu_Job
Tag	GK001
Barry	GK002
Paulo	GK002

<Job_Skills>

Stu_Skills	Stu_Job
Marketing	GK001
PR	GK002
Graphic Designing	GK003

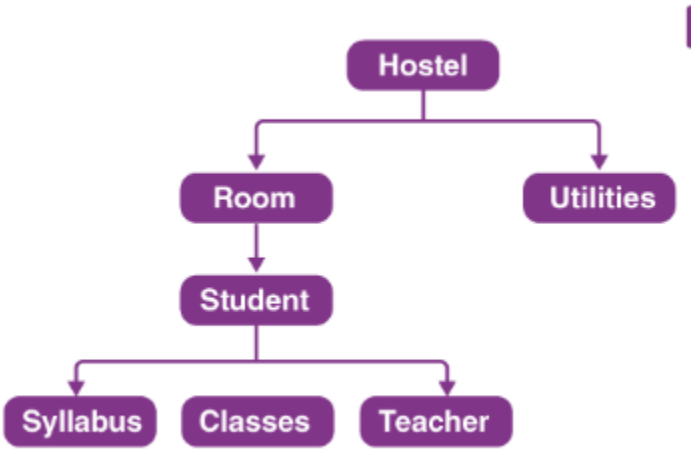
Our Join Dependency would be:

{(Stu_Name, Stu_Skills), (Stu_Name, Stu_Job), (Stu_Skills, Stu_Job)}

The relations given above have join dependency. Thus, they do not happen to be in 5NF. It means that the join relation of the three relations given above is equal to the very original relation <Student>.

Example 2

Let us consider some special classes of join dependencies that help us in capturing data dependencies that are present in a data structure that is hierarchical in nature.



This hierarchical organisation informs the reader about the rooms, and the students currently living in the room depend only on the hostel but not the utilities present in that hostel. Since hostels have multiple rooms, FDs are NOT adequate when we want to describe the data dependency among hostels and rooms or utilities.

In such a case, the multivalued dependencies,

Hostel ->-> room or

Hostel ->-> utilities hold

Thus, using the first-order hierarchical decomposition, one would be enabled to represent data dependencies that are present in a hierarchical data structure in a natural way. Thus, one can store the hostel database as a lossless join of the following:

Hostel_utility (hostel, utilities),

Hostel_room (hostel, room, student, syllabus, classes, teacher)

ACID PROPERITIES

What are ACID Properties in DBMS?

Transactions refer to the single logical units of work that access and (possibly) modify the contents present in any given database. We can access the transactions using the read and write operations.

If we want to maintain database consistency, then certain properties need to be followed in the transactions known as the ACID (Atomicity, Consistency, Isolation, Durability) properties. Let us discuss them in detail.